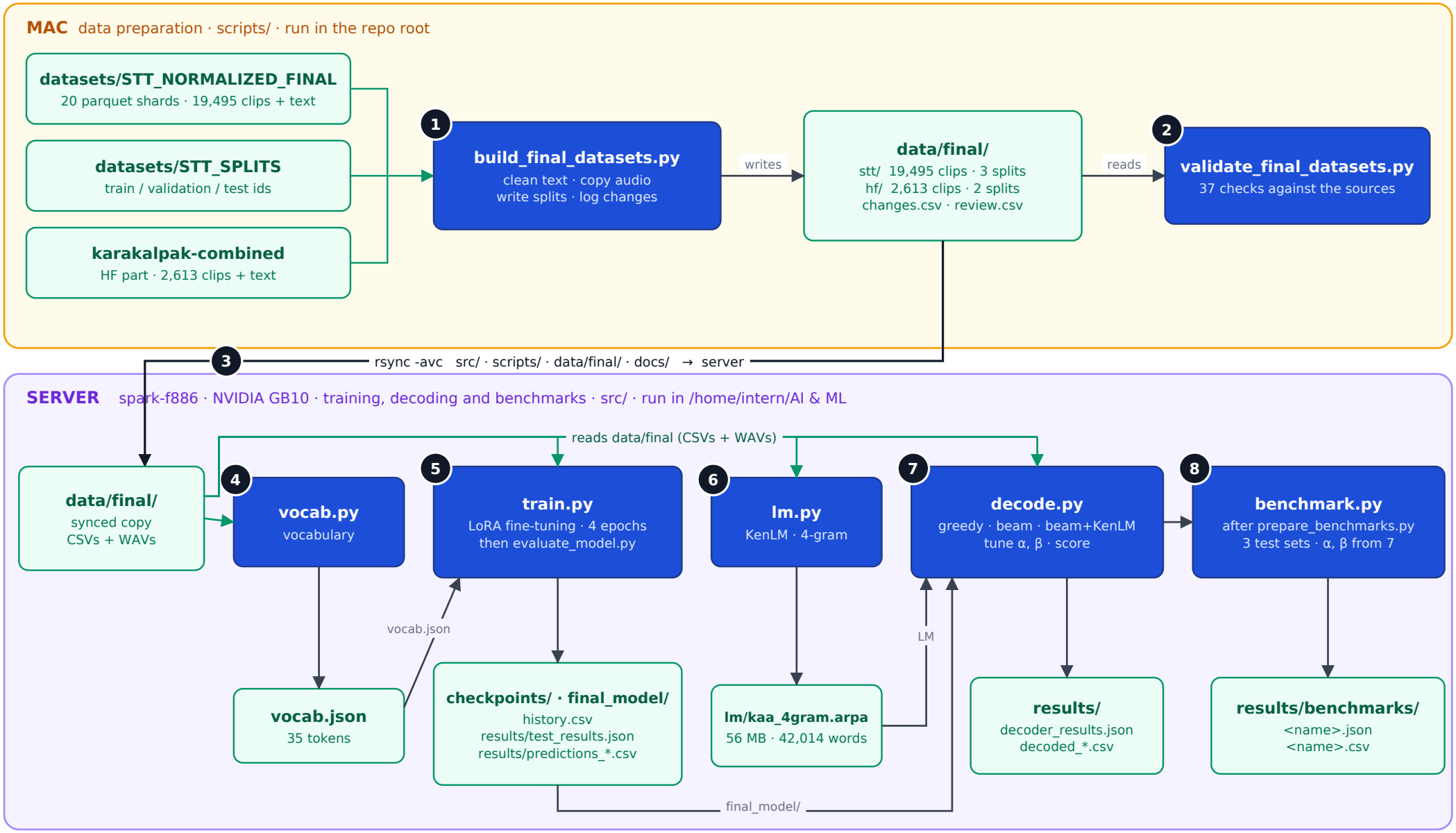
End-to-end pipeline

What runs where, in order. The numbers match the steps in the walkthrough PDF.

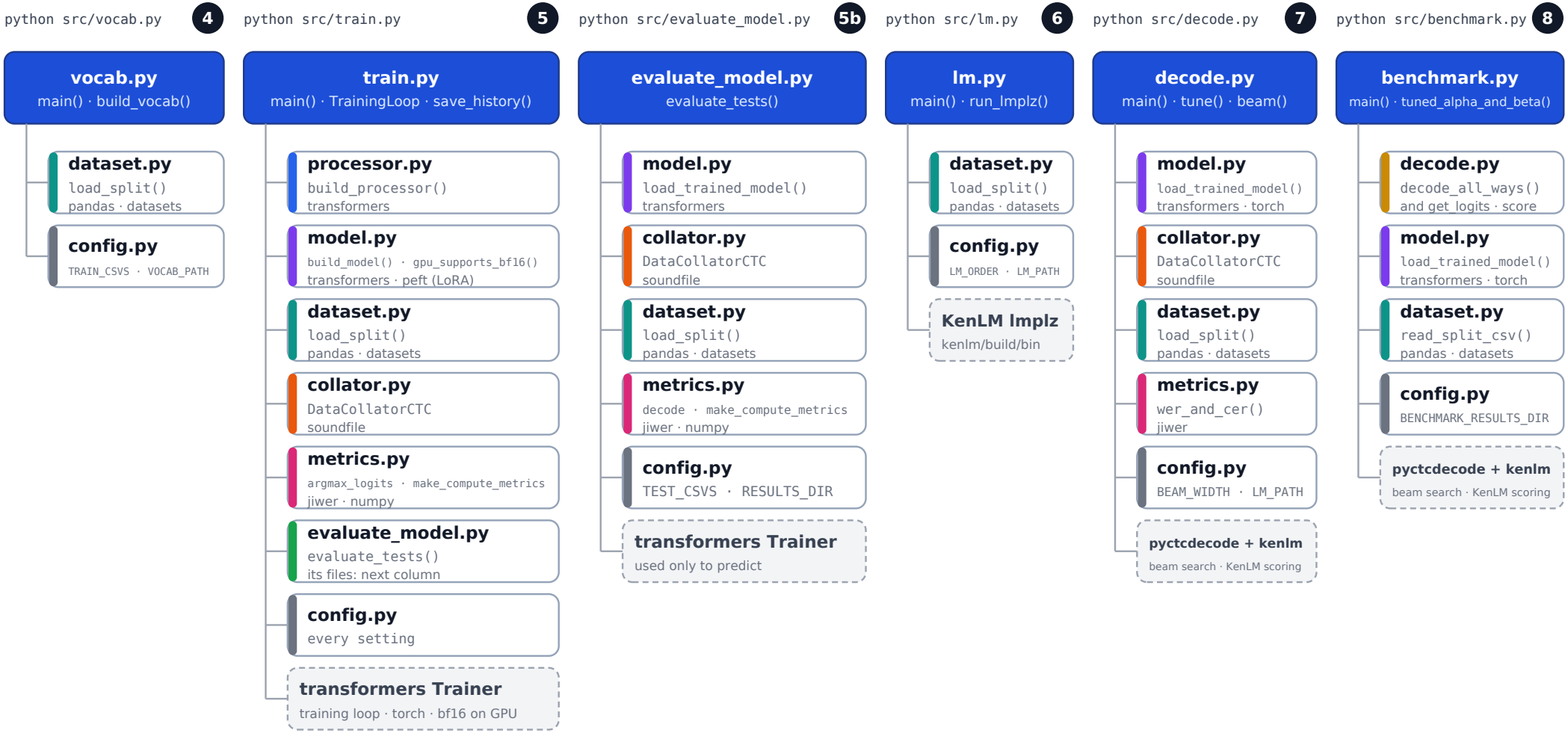


lm.py needs only the train text, so step 6 can run any time; decode.py needs the trained model and the LM; benchmark.py also needs decode.py's α and β.

script you run · data / files · 1 step number · reads / writes

Which files each command uses

Each column is one command. Under it: the src/ files it imports (coloured by file) and the outside tools it needs.



dataset.py, collator.py, metrics.py and model.py are shared by several commands; every file reads config.py.
Step 8 also needs scripts/prepare_benchmarks.py, run once before it to build data/benchmarks/.

Files: config.py · dataset.py · collator.py · metrics.py · processor.py · model.py · evaluate_model.py · decode.py

Inside train.py

What main() does, in order, and what happens inside train.train().

main()

1. `build_processor()`
   processor.py · tokenizer from vocab.json + feature extractor
2. `build_model(processor)`
   model.py · XLS-R 300M + new CTC head (35) + LoRA r=32
3. `load_split(TRAIN_CSVS / VALIDATION_CSVS)`
   dataset.py · 17,685 train · 1,950 validation clips
4. `TrainingLoop(args=training_arguments(), ...)`
   4 epochs · 8 clips × 4 = 32 per step · LR 5e-4 · bf16
5. `train.train(latest_checkpoint())`
   2,212 steps · the loop on the right · resumes if stopped
6. `save_history(log_history)`
   history.csv · WER/CER per epoch
7. `save_final_model(train, processor)`
   LoRA merged into the weights → final_model/
8. `evaluate_tests(final_model)`
   evaluate_model.py · test WER/CER → test_results.json

train.train()

repeats for 2,212 steps (553 per epoch × 4 epochs)

every batch of 8 clips

- DataLoader (8 workers) picks clips of similar length
- DataCollatorCTC: audio_batch() + label_batch()
- forward pass: XLS-R + LoRA → 35 scores per 20 ms frame
- CTC loss → backward (only LoRA + CTC head change)

every 4 batches (32 clips)

- optimizer step · learning-rate schedule

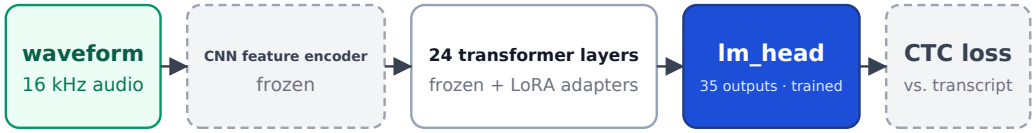
end of each epoch

- evaluate on 1,950 validation clips: argmax_logits → compute_metrics → decode() → wer_and_cer()
- TrainingLoop._save → checkpoints/checkpoint-N

after the last epoch

- reload the checkpoint with the lowest validation WER

The model a batch goes through



LoRA (rank 32) is added to q_proj, k_proj, v_proj, out_proj, intermediate_dense and output_dense in every layer: 14,191,651 trainable of 329,666,246 parameters (4.3%).

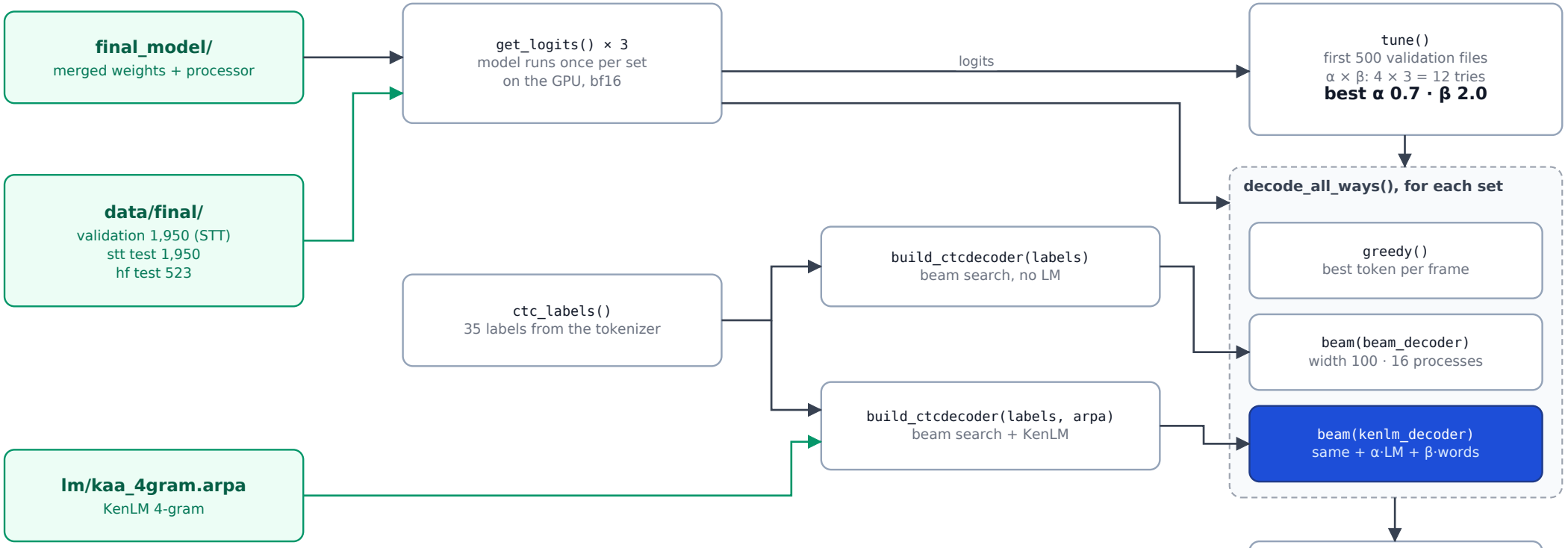
Validation after each epoch (1,950 STT clips, greedy decoding)

epoch	step	WER	CER	loss
1	553	85.88%	25.08%	0.948
2	1,106	55.89%	13.07%	0.515
3	1,659	42.77%	9.65%	0.411
4	2,212	**38.40%**	**8.72%**	0.381

Run time on the server: 2 h 31 min for 2,212 steps; the best epoch (4) became final_model/.

Inside decode.py

The trained model runs once per set; the same logits are then decoded three ways and scored.



Results: WER % / CER %

	greedy	beam	beam + KenLM
validation (1,950)	38.35 / 8.71	37.56 / 8.51	**18.97 / 5.93**
stt test (1,950)	38.37 / 8.90	37.75 / 8.73	**18.61 / 5.93**
hf test (523)	22.32 / 3.65	21.68 / 3.53	**18.30 / 3.08**

KenLM halves WER on STT (38.4% → 18.6%); on HF it drops from 22.3% to 18.3%.

Run time on the server: about 4 minutes.

Shallow fusion: how a beam hypothesis is scored

`score = log P_ctc(text | audio) + α · log P_KenLM(text) + β · words`

α = weight of the language model · β = bonus per word